

Lab5: Task

Process and Thread

1. Process

A. Fork()

對於 child 本身而言，其 pid 為 0，parent 為非 0。

pid_t:

child 會從 fork() 調用的地方開始執行，但回傳值為 0，這樣 child 就可以知道它自己是 child。

parent process：fork() 會在 parent 中回傳 child 的 process ID

parent 會收到一個大於 0 的回傳值，這個值是新創建的 child process ID

```
Parent pid vs child pid
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h> // use for wait()

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
    } else if (pid == 0) {
        // child process
        printf("This is the child process\n");
        printf("Child process PID: %d, Parent PID: %d\n", getpid(), getppid());
    } else {
        // parent process
        wait(NULL); // parent will wait for child to finish
        printf("This is the parent process, child PID is %d\n", pid);
        printf("Parent process PID: %d\n", getpid());
    }

    return 0;
}
```

B. Harvest:wait() and waitpid()

簡單來說，在 child process 結束時，會調用 exit(回傳值)或是 exit(0)，前者回傳給 parent，後者不回傳值並在結束後直接砍掉自己。有關回傳的部分，因為 process 間的 address space 不共用，因此在 parent process 中需要調用 wait 進行收割並解讀 (WEXITSTATUS)。具體來說 harvest 又分為 wait()與 waitpid()

- wait()

函數會讓 parent process 暫停執行，直到任意一個 child process 終止為止。當 child process 結束時，wait() 函數會回收 child process 的資源 (killing zombie process)。如果 parent process 有多個 child process，wait() 會等待其中任意一個 child process 終止，並返回該 child process 的 ID (PID)。當 parent process 不關心哪個 child process 終止，只要任意一個 child process 結束，就使用 wait()

- waitpid()

waitpid() 是一個功能更靈活的函數，允許 parent process 根據特定的 PID 來等待指定的 child process。parent process 可以使用 waitpid() 指定等待的 child process，並且可以通過選項修改它的行為，比如不阻塞進程或等待其他狀態變化 (如暫停、繼續執行等)。

選項：

WNOHANG：如果沒有 child process 已經終止，則 waitpid() 會立即返回，而不是阻塞。

WUNTRACED：等待因信號暫停的 child process。

WCONTINUED：等待已經恢復執行的 child process。

```
/*
 * process.c
 */
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
/* Errors */
/* Input/Output */
/* General Utilities */
#include <sys/types.h> /* Primitive System Data Types */
#include <sys/wait.h> /* Wait for Process Termination */
#include <unistd.h>

/* Symbolic Constants */
pid_t childpid; /* variable to store the child's pid */

void childfunc(void)
{
    int retval;
    /* user-provided return code */
    printf("CHILD: I am the child process!\n");
    printf("CHILD: My PID: %d\n", getpid());
    printf("CHILD: My parent's PID is: %d\n", getppid());
    printf("CHILD: Sleeping for 1 second...\n");
    sleep(1); /* sleep for 1 second */
    printf("CHILD: Enter an exit value (0 to 255): ");
    scanf("%d", &retval);
    printf("CHILD: Goodbye!\n");
    exit(retval); /* child exits with user-provided return code */
}

void parentfunc(void)
{
    int status; /* child's exit status */

    printf("PARENT: I am the parent process!\n");
    printf("PARENT: My PID: %d\n", getpid());
    printf("PARENT: My child's PID is %d\n", childpid);
    printf("PARENT: I will now wait for my child to exit.\n");

    /* wait for child to exit, and store its status */
    wait(&status);
    printf("PARENT: Child's exit code is: %d\n", WEXITSTATUS(status));
    printf("PARENT: Goodbye!\n");

    exit(0); /* parent exits */
}

int main(int argc, char *argv[])
{
    /* now create new process */
    childpid = fork();

    if (childpid >= 0) { /* fork succeeded */
        if (childpid == 0) { /* fork() returns 0 to the child process */
            childfunc();
        } else { /* fork() returns new pid to the parent process */
            parentfunc();
        }
    } else { /* fork returns -1 on failure */
        perror("fork"); /* display error message */
        exit(0);
    }
}
```

```
    return 0;
}
```

```
/*
 * waitpid.c -- shows how to get child's exit status
 */

#include <errno.h>      /* Errors */
#include <stdio.h>      /* Input/Output */
#include <stdlib.h>     /* General Utilities */
#include <sys/types.h>  /* Primitive System Data Types */
#include <sys/wait.h>   /* Wait for Process Termination */
#include <time.h>       /* Time functions */
#include <unistd.h>     /* Symbolic Constants */

pid_t childpid; /* variable to store the child's pid */

void childfunc(void)
{
    int randtime; /* random sleep time */
    int exitstatus; /* random exit status */

    printf("CHILD: I am the child process!\n");
    printf("CHILD: My PID: %d\n", getpid());

    /* sleep */
    srand(time(NULL));
    randtime = rand() % 5;
    printf("CHILD: Sleeping for %d second...\n", randtime);
    sleep(randtime);

    /* rand exit status */
    exitstatus = rand() % 2;
    printf("CHILD: Exit status is %d\n", exitstatus);

    printf("CHILD: Goodbye!\n");
    exit(exitstatus); /* child exits with user-provided return code */
}

void parentfunc(void)
{
    int status; /* child's exit status */
    pid_t pid;

    printf("PARENT: I am the parent process!\n");
    printf("PARENT: My PID: %d\n", getpid());

    printf("PARENT: I will now wait for my child to exit.\n");

    /* wait for child to exit, and store its status */
    do
    {
        pid = waitpid(childpid, &status, WNOHANG);
        printf("PARENT: Waiting child exit ...\n");
        sleep(1);
    }while (pid != childpid);

    if (WIFEXITED(status)) {
        // child process exited normally.
        printf("PARENT: Child's exit code is: %d\n",
            WEXITSTATUS(status));
    }else{
        // Child process exited thus exec failed.
        // LOG failure of exec in child process.
    }
}
```

```

    printf("PARENT: Child process executed but exited failed.\n");
}

printf("PARENT: Goodbye!\n");
exit(0); /* parent exits */
}

int main(int argc, char *argv[])
{
    /* now create new process */
    childpid = fork();

    if (childpid >= 0) { /* fork succeeded */
        if (childpid == 0) { /* fork() returns 0 to the child process */
            childfunc();
        } else { /* fork() returns new pid to the parent process */
            parentfunc();
        }
    } else { /* fork returns -1 on failure */
        perror("fork"); /* display error message */
        exit(0);
    }
    return 0;
}

```

C. nice()

nice 為一個 process 的 priority 的優先值，值越大優先度越低。nice() 會加上 increment 加到自己身上更新自己的 priority，這類型的舉動會需要 root 權限。

```

#include <stdio.h>
#include <unistd.h>

int main() {
    int current_nice;
    int increment = -10; // nice value minus 10

    // get current nice value using nice(0)
    current_nice = nice(0);
    printf("Current nice value: %d\n", current_nice);

    // change nice value
    if (nice(increment) == -1) {
        perror("nice");
        return 1;
    }

    current_nice = nice(0);
    printf("New nice value after adjustment: %d\n", current_nice);

    return 0;
}

```

D. exec()

exec 系列函數是用來在當前進程中執行另一個 process 的。當使用 exec 函數後，目前的 process 的程式碼會被新的程式碼所取代，但 process ID (PID) 不變。這意味著，exec 不會創建新的 process，它只會用新程式取代當前 process 的程式碼。常見的 exec 系列有：

1. 使用 execl 執行 ls 命令，指定完整路徑 "/bin/ls"


```
execl("/bin/ls", "/bin/ls", "-l", "-r", "-t", NULL);
```
2. 使用 execlp 執行 ls 命令，這裡不需要指定完整路徑，因為 execlp 會查找

PATH 中的路徑

```
execlp("ls", "ls", "-l", NULL);
```

3. 使用 `execv`，傳遞引數是以陣列的形式，並指定完整路徑 `"/bin/ls"`

```
char *arg1[] = {"/bin/ls", "-l", NULL};
```

```
execv(arg1[0], arg1);
```

4. 使用 `execvp`，傳遞引數是以陣列的形式，但這裡不用指定完整路徑，因為 `execvp` 會從 PATH 中查找

```
char *arg2[] = {"ls", "-l", NULL};
```

```
execvp(arg2[0], arg2);
```

```
/*
 * exec()
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        /* child process */
        /* Use execl to run "ls" command */
        execl("/bin/ls", "ls", "-l", NULL);
        perror("execl failed");
        exit(EXIT_FAILURE);
    } else {
        /* parent process */
        printf("Parent process (PID: %d) waiting for child process (PID: %d)\n",
            getpid(), pid);

        /* waiting for child process */
        wait(NULL);
        printf("Child process finished.\n");
    }

    return 0;
}
```

執行結果:

```
wang@EOS:~/Desktop/lab5/handout$ ./exec_example
Parent process (PID: 6327) waiting for child process (PID: 6328)
total 192
-rwxrwxr-x 1 wang wang 16136 + 18 17:28 exec
-rw-rw-r-- 1 wang wang 745 + 18 17:28 exec.c
-rwxrwxr-x 1 wang wang 16264 + 19 21:10 exec_example
-rw-rw-r-- 1 wang wang 787 + 19 21:10 exec_example.c
-rwxrwxr-x 1 wang wang 16176 + 18 16:11 fork1
-rw-rw-r-- 1 wang wang 681 + 18 16:11 fork1.c
-rwxrwxr-x 1 wang wang 16048 + 18 17:15 nice
-rw-rw-r-- 1 wang wang 450 + 18 17:15 nice.c
-rwxrwxr-x 1 wang wang 16456 + 18 16:12 process_control
-rw-rw-r-- 1 wang wang 1809 + 18 16:12 process_control.c
-rwxrwxr-x 1 wang wang 16488 + 18 17:04 process_control_waitpid
-rw-rw-r-- 1 wang wang 2523 + 18 17:04 process_control_waitpid.c
-rwxrwxr-x 1 wang wang 16160 + 18 18:14 pthread
-rw-rw-r-- 1 wang wang 677 + 18 18:14 pthread.c
-rwxrwxr-x 1 wang wang 16312 + 18 22:30 pthread_detach
-rw-rw-r-- 1 wang wang 1220 + 19 01:11 pthread_detach.c
-rwxrwxr-x 1 wang wang 16472 + 18 18:26 pthread_join
-rw-rw-r-- 1 wang wang 1396 + 18 18:16 pthread_join.c
Child process finished.
```

E. function pointer

```
/*
function_ptr.c
*/
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

void perform_operation(int (*operation)(int, int), int x, int y) {
    int result = operation(x, y);
    printf("Result: %d\n", result);
}

int main() {
    int (*func_ptr)(int, int);

    func_ptr = add;
    printf("Using add function:\n");
    perform_operation(func_ptr, 5, 3);

    func_ptr = subtract;
    printf("Using subtract function:\n");
    perform_operation(func_ptr, 5, 3);

    return 0;
}
```

F. Zombie process

當一個 child process 結束執行時，它的狀態會變為 "zombie"，這意味著它已經終止，但其 parent process 尚未調用 `wait()` 或 `waitpid()` 來獲取該 child process 的退出狀態和釋放資源。這種情況會導致該 child process 的進程控制塊 (Process Control Block, PCB) 仍然存在於系統中，並且 process ID (PID) 也不會被重用。

```
/*
*zombie test program
*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    if ((pid = fork()) < 0) {
        perror("fork");
        exit(1);
    }

    /* Child */
    if (pid == 0)
        exit(0);

    /* Parent
     * Gives you time to observe the zombie using ps(1) ... */
    sleep(100);

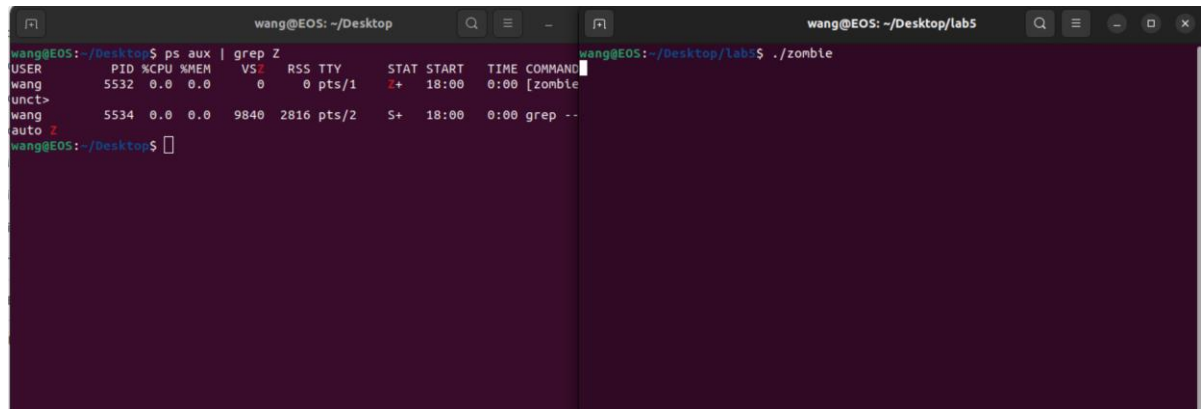
    /* ... and after that, parent wait(2)s its child's

```

```
    * exit status, and prints a relevant message. */
    pid = wait(&status);
    if (WIFEXITED(status))
        fprintf(stderr, "\n\t[%d]\tProcess %d exited with status %d.\n",
                (int) getpid(), pid, WEXITSTATUS(status));

    return 0;
}
```

執行 `ps aux | grep Z`



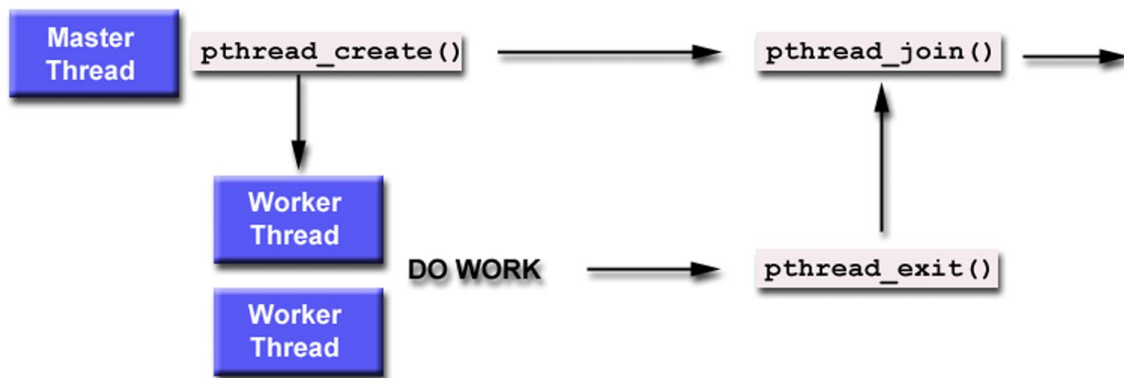
The image shows two terminal windows side-by-side. The left window, titled 'wang@EOS: ~/Desktop', displays the output of the command 'ps aux | grep Z'. The output is a table with columns: USER, PID, %CPU, %MEM, VSZ, RSS, TTY, STAT, START, TIME, and COMMAND. It shows two entries for the user 'wang': one with PID 5532 and another with PID 5534. The right window, titled 'wang@EOS: ~/Desktop/lab5', shows the command './zombie' being executed, with no visible output.

```
wang@EOS: ~/Desktop$ ps aux | grep Z
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
wang      5532  0.0  0.0      0      0 pts/1    Z+   18:00   0:00 [zombie]
wang      5534  0.0  0.0  9840  2816 pts/2    S+   18:00   0:00 grep --

wang@EOS: ~/Desktop$
```

```
wang@EOS: ~/Desktop/lab5$ ./zombie
```

2. Thread



A. pthread_create()

show 是一個 function pointer，接收一個指向 void 的 pointer 作為參數。threadid 型別為 long。pthread_t threads[5] 用於儲存每個 thread id。

rc = pthread_create(&threads[t], NULL, show, (void *)t);
rc 用於儲存 pthread_create() 的回傳值，&threads[t] 新創建執行緒的 ID 存儲位置，NULL 為屬性，例如 joinable, detachable，第三個參數傳入 function ptr，第四個部分為傳入 function 參數，如果超過一個參數就用 struct 指標傳，在這邊則是把 t 轉成 void 指針。

```
/*
 * pthread.c -- shows how to create a thread
 */

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 5

void *show(void *threadid)
{
    long tid;
    tid = (long)threadid;
    printf("Hello! I am thread #%ld!\n", tid);

    pthread_exit(NULL);
}

int main(int argc, char *argv[])
{
    pthread_t threads[5];
    int rc;
    int t;

    for(t=0; t<NUM_THREADS; t++){
        printf("In main(): creating thread %d\n", t);
        rc = pthread_create(&threads[t], NULL, show, (void *)t);
        if (rc){
            // nonzero means error
            printf("ERROR; pthread_create() returns %d\n", rc);
            exit(-1);
        }
    }

    printf("Main: program completed. Exiting.\n");
    pthread_exit(NULL);
}
```

執行結果

```
wang@EOS:~/Desktop/lab5/handout$ ./pthread
In main(): creating thread 0
In main(): creating thread 1
In main(): creating thread 2
Hello! I am thread #0!
In main(): creating thread 3
Hello! I am thread #1!
Hello! I am thread #2!
In main(): creating thread 4
Main: program completed. Exiting.
Hello! I am thread #3!
Hello! I am thread #4!
```

B. attach vs. detach

attach 使用 join，代表 master thread 在 worker thread 結束後會回收 worker thread 的資料，使用 pthread_exit() 再 pthread_join()。detach 則為在 worker thread 結束後不需要回傳值，並且可以直接被系統回收資源。簡單來說，pthread_detach 用於將一個執行緒標記為分離狀態，這意味著當該 thread 結束時，其佔用的資源將自動被系統回收。

在以下程式中，使用的是 attach 模式，在 worker thread 結束後用 exit 回傳 void 指針，並在 master thread 用 pthread_join() 收資料，特別注意指針型態。

```
/*
 * join.c -- shows how to "wait" for thread completions
 * attach
 */

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUM_THREADS 4

void *BusyWork(void *t)
{
    long tid;
    int *result = malloc(sizeof(int));

    tid = (long)t;
    printf("Thread %ld starting...\n", tid);

    *result = 1 + 1;

    printf("Thread %ld done. Result = %d\n", tid, *result);
    /* return result to master thread */
    pthread_exit((void *)result); // beware of return type
}

int main (int argc, char *argv[])
{
    pthread_t thread[NUM_THREADS];
    pthread_attr_t attr;
    int rc;
    long t;
    void *result; // receive value from worker thread
```

```

/* set attribute and joinable*/
pthread_attr_init(&attr);
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_JOINABLE);

for (t = 0; t < NUM_THREADS; t++) {
    printf("Main: creating thread %ld\n", t);
    rc = pthread_create(&thread[t], &attr, BusyWork, (void *)t);
    if (rc) {
        printf("ERROR; pthread_create() returns %d\n", rc);
        exit(-1);
    }
}

pthread_attr_destroy(&attr);
for (t = 0; t < NUM_THREADS; t++) {
    rc = pthread_join(thread[t], &result);
    if (rc) {
        printf("ERROR; pthread_join() returns %d\n", rc);
        exit(-1);
    }
    printf("Main: join with thread %ld (result from worker: %d)\n", t, *(int *)result); // same address space
    free(result);
}

printf("Main: program completed. Exiting.\n");
pthread_exit(NULL);
}

```

以下部分則為 detach

```

/*
 * deteach.c -- shows how to detach a thread
 */

#include <errno.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void *threadfunc(void *parm)
{
    printf("Inside secondary thread\n");
    sleep(3);
    printf("Exit secondary thread\n");
    pthread_exit(NULL);
}

int main(int argc, char **argv)
{
    pthread_t thread[0];
    int rc = 0;

    printf("Create a thread using attributes that allow detach\n");
    rc = pthread_create(&thread[0], NULL, threadfunc, NULL);
    if (rc) {
        printf("ERROR; pthread_create() returns %d\n", rc);
        exit(-1);
    }

    printf("Detach the thread after it terminates\n");
}

```

```

rc = pthread_detach(thread[0]);
if (rc) {
    printf("ERROR; pthread_detach() returns %d\n", rc);
    exit(-1);
}

printf("Detach the thread again\n");
rc = pthread_detach(thread[0]);
/* EINVAL: No thread could be found corresponding to that
 * specified by the given thread ID.
 */
if (rc != EINVAL) {
    printf("Got an unexpected result! rc=%d\n", rc);
    exit(1);
}
printf("Second detach fails as expected.\n");

/* sleep() is not a very robust way to wait for the thread */
sleep(6);
printf("Main() completed.\n");
return 0;
}

```

pthread example

```

/*
 *pthread example.c
 */
Master thread creates two worker threads
one for calculating 20 + 30
one for calculating 20 - 30

then after pthread_join()
in master thread, calculating (20 + 30)*(20 - 30)
*/

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int a;
    int b;
    int result;
    char operation; // '+' or '-'
} ThreadData;

void *calculate(void *data) {
    ThreadData *threadData = (ThreadData *)data;

    if (threadData->operation == '+') {
        threadData->result = threadData->a + threadData->b;
    } else if (threadData->operation == '-') {
        threadData->result = threadData->a - threadData->b;
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t threads[2];
    ThreadData threadData[2]; //store each thread's data
    int rc;

    threadData[0].a = 20;
    threadData[0].b = 30;
    threadData[0].operation = '+';
    rc = pthread_create(&threads[0], NULL, calculate, (void *)&threadData[0]);
    if (rc) {
        printf("ERROR; pthread_create() returns %d\n", rc);
        exit(-1);
    }
}

```

```

}

threadData[1].a = 20;
threadData[1].b = 30;
threadData[1].operation = '-';
rc = pthread_create(&threads[1], NULL, calculate, (void *)&threadData[1]);
if (rc) {
    printf("ERROR; pthread_create() returns %d\n", rc);
    exit(-1);
}

/* use pthread join to collect data */
for (int t = 0; t < 2; t++) {
    rc = pthread_join(threads[t], NULL);
    if (rc) {
        printf("ERROR; pthread_join() returns %d\n", rc);
        exit(-1);
    }
}

int finalResult = threadData[0].result * threadData[1].result;
printf("Result of (20 + 30) * (20 - 30) = %d * %d = %d\n",
        threadData[0].result, threadData[1].result, finalResult);

printf("Main: program completed. Exiting.\n");
pthread_exit(NULL);
}

```

3. Lab5 hondout

A. signal()

- SIGNIT

當輸入 ctrl+c , 會產生一個 SIGNIT 信號 , signal()會執行 server_handler 內容。

```
/* Close socket when catching SIGINT signal ( Signal Interrupt such as Ctrl + C )
*/
void server_handler(int signum) {
    close(sockfd);
}
int main(int argc, char* argv[]){
    signal(SIGINT, server_handler);
}
```

- SIGCHLD

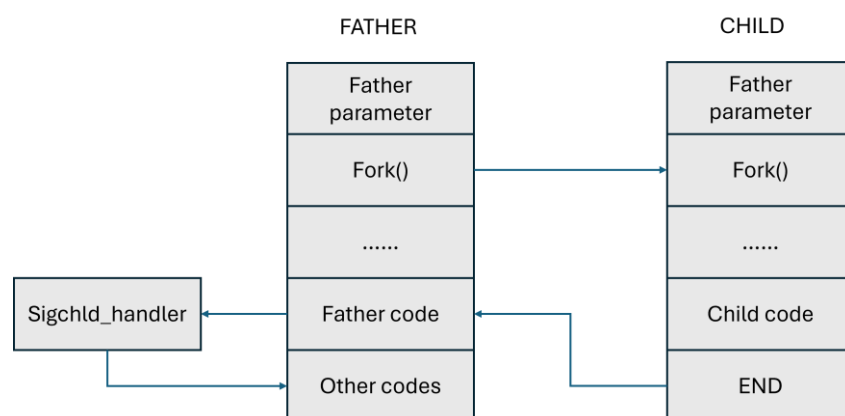
以下為一個範例 SIGCHLD signal , 會在 child process 結束後發送 SIGCHLD 給 parent process , parent 轉而執行 sigchld_handler , 執行完該函數後再繼續執行剩下內容。

```
/* SIGCHLD */
pid_t pid = 0;

void sigchld_handler(int sig) {
    printf("father call the SIGCHLD signal handler. num = %d\n", sig);
}

int main() {
    signal(SIGCHLD, sigchld_handler);
    pid = fork();

    if (pid == -1) {
        exit(1);
    } else if (pid == 0) { // child code
        printf("child process is running, pid = %d.\n", getpid());
    } else { // father code
        pause();
        printf("father process runs again.\n");
    }
    return 0;
}
```



需要注意的是 , 如果 father 先結束 , sigchld_handler() 也不會執行因為 father 已經結束 , 就會形成 zombie process , 最簡單的解決辦法就是用 sleep() 延緩 father 的執行時間 , 比較進階的辦法則是在 sigchld_handler() 使用 wait() 或 waitpid() 。以下範例

```
/*sigchld_handler with wait pid */
```

```

pid_t fd1 = 0;
pid_t fd2 = 0;
int stat = 0;

int wd1 = 0;
int wd2 = 0;

void sigchld_handler(int sig) {
    // represent if this function is called.
    printf("father call the SIGCHLD signal handler.\n");

    int wd1 = waitpid(fd1, &stat, 0);
    int wd2 = waitpid(fd2, &stat, 0);

    if (wd1 == fd1) { // child1 finish
        printf("child1 process has finished.\n");
    }

    if (wd2 == fd2) { // child2 finish
        printf("child2 process has finished.\n");
    }
}

int main() {
    signal(SIGCHLD, sigchld_handler);

    int fd1 = fork();
    if (fd1 == -1) {
        exit(1);
    } else if (fd1 == 0) { // child1 code
        printf("child1 process is running, pid = %d.\n", getpid());
    } else {
        int fd2 = fork();
        switch (fd2) {
            case -1:
                exit(1);
                break;
            case 0: // child2 code
                printf("child2 process is running, pid = %d.\n", getpid());
                break;
            default: // father code
                pause();
                printf("father process runs again.\n");
                break;
        }
    }

    return 0;
}

```

或者如 Lab5 handout 中以下方式

```

/* zombie_handler */

void zombie_handler(int signum) {
    while (waitpid(-1, NULL, WNOHANG) > 0);
}

int main(int argc, char* argv[]){
    signal(SIGCHLD, zombie_handler);
}

```

B. Dup2

在 Unix 系統中，每個 process 預設會有三 file description，0 代表標準輸入 (STDIN_FILENO)、1 代表標準輸出 (STDOUT_FILENO)、2 代表標準錯誤輸出 (STDERR_FILENO)。文件描述符可以在 /proc/process ID/fd/ 目錄下查看。dup2 是一個 Unix 系統調用，用來將一個檔案描述符複製到另一個檔案描述符。簡單來

說，`dup2(oldfd, newfd)` 會將 `newfd` 關閉，然後將 `oldfd` 複製到 `newfd`，並返回 `newfd`。之後，`newfd` 和 `oldfd` 會指向相同的檔案/資源，並共享相同的狀態（如文件偏移量等）。如果 `oldfd` 和 `newfd` 是一樣的，那麼什麼都不會發生。使用場景通常是在程序中重新定向輸出或錯誤輸出。常見的應用是將標準輸出（`stdout`）或標準錯誤輸出（`stderr`）重定向到一個檔案或是新的 file discription。

```
/* dup2 */

#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>

int main() {
    int file_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (file_fd < 0) {
        perror("open");
        return 1;
    }

    // use dup2 to redirect (fd = 1) to file_fd
    // STDOUT_FILENO means standard output with value 1
    if (dup2(file_fd, STDOUT_FILENO) < 0) {
        perror("dup2");
        return 1;
    }

    printf("This message will be written to output.txt!\n");
    close(file_fd);

    return 0;
}
```

C. Basic socket programming in C

- `int server_fd = socket(int domain, int type, int protocol)`

`domain` 決定 `ipv4`, `ipv6`，`type` 決定 `tcp` 或 `udp`，`protocol` 通常為 0 表預設。若成功會回傳 `socket file discriptor`，失敗則回傳 -1。

```
/* Basic socket server */

#include <stdio.h>
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    // AF_INET = IPv4
    // SOCK_DGRAM = UDP
    int socket_fd = socket(AF_INET, SOCK_DGRAM, 0);

    // if error, return value is -1
    if (socket_fd < 0) {
        printf("Fail to create a socket.");
    }

    close(socket_fd);
    return 0;
}
```

- `sockaddr`

`sockaddr` 是 `socket` 的通用地址結構，其定義如下：

```

struct sockaddr {
    unsigned short int sa_family; // 2 bytes
    char sa_data[14];           // 14 bytes
};

```

後續為了提高可讀性，另外定義了 `sockaddr_in` 結構用來存取網路相關資料，`sockaddr_in6` 則是定義了 IPV4 相關地址。

```

struct sockaddr_in {
    // sa_family_t sin_family
    unsigned short int sin_family; // 2 bytes
    unsigned short int sin_port;   // 2 bytes
    struct in_addr sin_addr;       // 4 bytes
    unsigned char sin_zero[8];     // 填充，讓 sockaddr_in 的 size 跟 sockaddr 相同
};

```

- `bind`

這個函式可以讓前面創建的 socket 實際綁定到本機的某個 port 上面，這樣子 client 端在送資料到某個 port 的時候，我們寫的 server 程式才可以在那個 port 上面運行，處理資料。

```

bind(int sockfd, struct sockaddr *addr, unsigned int addrlen)

```

`bind` 成功回傳 0，失敗回傳 -1。以下為一個 `sockaddr_in` 與 `bind` 範例

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 8080
#define MAX_CLIENTS 5
#define BUFFER_SIZE 1024

int main() {
    struct sockaddr_in address;

    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);

    // start binding
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Bind failed");
        exit(EXIT_FAILURE);
    }

    // start listening
    if (listen(server_fd, 3) < 0) {
        perror("Listen failed");
        exit(EXIT_FAILURE);
    }
}

```

- **setsocket**

以下方法允許重複使用相同 address，通常發生在剛斷開連線時會有幾秒鐘的“Address already in use”。

```
int opt = 1;
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))) {
    perror("setsockopt");
    exit(EXIT_FAILURE);
}
```

- **accept()**

用於在 server 端接收連線，原形如下：

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

使用上特別注意 addrlen 是一個 socklen_t 的指標，可配合下面方法

```
struct sockaddr_in address;
socklen_t addrlen = sizeof(address);

. . . . .

int client_fd = accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```